Introducción a la programación con Python: Guía rigurosa basada en la documentación oficial de Python — sin variables, tipos, condicionales ni funciones

Edison Achalma y Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Tabla de contenidos

Introduction **3**

1 Introducción a la programación con Python **3**
1.1 Tabla de contenidos . . . 3
1.2 Antes de empezar . . . 4
1.2.1 Motivación . . . 4
1.2.2 Intuición . . . 4
1.2.3 Modelo mental . . . 4
1.3 1. ¿Qué es Python? Definición oficial . . . 4
1.4 2. ¿Por qué existe Python? La motivación original . . . 5
1.5 3. Características que distinguen a Python como lenguaje . . . 5
1.6 4. Por qué los programas en Python son compactos y legibles . . . 5
1.7 5. Extensibilidad: Python como lenguaje de extensión . . . 6
1.8 6. Origen del nombre “Python” . . . 6
1.9 7. Estructura oficial del tutorial de Python . . . 6
1.10 8. Versiones de Python y ciclo de vida oficial . . . 7
1.11 9. Instalación de Python: documentación oficial . . . 7
1.11.1 9.1. Dónde descargar . . . 7
1.11.2 9.2. Windows . . . 7
1.11.3 9.3. Verificación de firma digital en Windows y macOS . . . 7
1.11.4 9.4. Linux: nota sobre el alcance de esta guía . . . 8

Edison Achalma https://orcid.org/0000-0001-6996-3364

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

1.12 10. Verificación de la integridad del instalador 8
1.13 Tu primer programa 8
1.13.1 Ejemplos 9
1.14 11. Invocar el intérprete: conceptos generales 9
1.14.1 11.1. Ubicación habitual del ejecutable 9
1.14.2 11.2. En Windows 10
1.14.3 11.3. Comportamiento general, similar al shell de Unix 10
1.15 12. El modo interactivo 10
1.16 13. Edición de línea de comandos en el intérprete 11
1.17 14. Ejecutar Python como script (modo no interactivo) 11
1.18 15. La opción `-c`: ejecutar un comando directo 11
1.19 16. La opción `-m`: ejecutar un módulo como script 11
1.20 17. La opción `-i`: modo interactivo después de un script 12
1.21 18. Paso de argumentos: `sys.argv` 12
1.22 19. El intérprete y su entorno: codificación del código fuente 13
1.22.1 19.1. Declarar una codificación distinta a la predeterminada 13
1.23 20. La línea shebang en Unix/Linux 13
1.24 21. Salir del intérprete 13
1.25 22. Comentarios en Python 14
1.26 23. El intérprete como calculadora: una vista previa controlada 14
1.27 24. PYTHONSTARTUP: personalizar el inicio del modo interactivo 15
1.28 25. Glosario mínimo para esta etapa 15
1.29 26. Qué sigue: hoja de ruta de las próximas guías 16
1.30 27. Referencias oficiales 16
1.31 Para cerrar 17
1.31.1 Errores comunes 17
1.31.2 Buenas prácticas 17
1.31.3 Ejercicio 17
1.31.4 Síntesis 17

2 Publicaciones Similares **18**

Introducción a la programación con Python

1 Introducción a la programación con Python

Esta guía es una introducción **pura** al lenguaje Python, construida exclusivamente sobre la documentación oficial: el *Python Tutorial* (docs.python.org/3/tutorial) y la página oficial de descargas (python.org/downloads). Trata **únicamente** qué es Python, su filosofía de diseño, cómo instalarlo y cómo invocar el intérprete (modo interactivo y modo script). **Deliberadamente no cubre** variables, expresiones, statements, objetos, ejecución condicional, iteraciones, funciones, dataframes ni machine learning — esos temas se abordan en guías posteriores y separadas, siguiendo tu propia secuencia ya establecida (02 a 11). Esta guía es, en ese sentido, el "capítulo 0": el terreno que hay que pisar antes de escribir la primera línea de código con sentido.

1.1 Tabla de contenidos

1. ¿Qué es Python? Definición oficial
2. ¿Por qué existe Python? La motivación original
3. Características que distinguen a Python como lenguaje
4. Por qué los programas en Python son compactos y legibles
5. Extensibilidad: Python como lenguaje de extensión
6. Origen del nombre "Python"
7. Estructura oficial del tutorial de Python
8. Versiones de Python y ciclo de vida oficial
9. Instalación de Python: documentación oficial
10. Verificación de la integridad del instalador
11. Invocar el intérprete: conceptos generales
12. El modo interactivo
13. Edición de línea de comandos en el intérprete
14. Ejecutar Python como script (modo no interactivo)
15. La opción `-c`: ejecutar un comando directo
16. La opción `-m`: ejecutar un módulo como script
17. La opción `-i`: modo interactivo después de un script
18. Paso de argumentos: `sys.argv`
19. El intérprete y su entorno: codificación del código fuente
20. La línea shebang en Unix/Linux
21. Salir del intérprete
22. Comentarios en Python
23. El intérprete como calculadora: una vista previa controlada
24. PYTHONSTARTUP: personalizar el inicio del modo interactivo
25. Glosario mínimo para esta etapa
26. Qué sigue: hoja de ruta de las próximas guías
27. Referencias oficiales

1.2 Antes de empezar

1.2.1 Motivación

Python es uno de los lenguajes más utilizados del mundo porque equilibra potencia con legibilidad. Puedes escribir desde un script de una línea hasta sistemas complejos de inteligencia artificial con la misma sintaxis clara. Aprender Python no es solo aprender un lenguaje: es abrir la puerta a automatización, análisis de datos, desarrollo web y mucho más.

1.2.2 Intuición

Imagina que le das instrucciones a un asistente muy inteligente pero literal: no interpreta, ejecuta exactamente lo que le dices. Python es ese asistente. Escribes órdenes en un archivo de texto con extensión `.py` y se las pasas al "intérprete" (el programa que entiende Python). El intérprete las lee línea por línea y las ejecuta.

Es como una receta de cocina: escribes los pasos en orden, y el intérprete los sigue al pie de la letra. Si un paso está mal escrito, la receta se detiene y te avisa.

1.2.3 Modelo mental

Python es un **lenguaje interpretado** (no compilado). Esto significa que:

1. **No necesitas compilar**: escribes código y lo ejecutas directamente.
2. **Ejecución línea por línea**: Python lee tu archivo de arriba a abajo.
3. **Tipado dinámico**: no declaras qué tipo de dato es cada variable; Python lo deduce.
4. **Indentación como sintaxis**: los espacios o tabulaciones no son solo estilo, **definen bloques de código**.

Lo más importante ahora: **todo en Python es un objeto** (incluyendo números, texto y funciones), y las **variables son etiquetas** que apuntas a esos objetos.

1.3 1. ¿Qué es Python? Definición oficial

Antes de instalar nada, conviene fijar con precisión qué es Python según su propia documentación, en vez de partir de descripciones de terceros.

El *Python Tutorial* oficial lo resume así: Python es, ante todo, un lenguaje pensado para **automatizar tareas** que surgen del trabajo cotidiano con computadoras — por ejemplo, hacer búsqueda y reemplazo sobre un gran número de archivos de texto, o renombrar y reorganizar un conjunto de archivos de fotos de forma compleja. También sirve para escribir una base de datos pequeña y personalizada, una aplicación gráfica (GUI) especializada, o un juego sencillo.

La documentación es explícita sobre dónde se ubica Python frente a otras alternativas: comparado con scripts de shell de Unix o archivos batch de Windows (buenos para mover archivos y cambiar datos de texto, pero no aptos para aplicaciones GUI o juegos) y comparado con programas en C/C++/Java (que pueden tardar mucho tiempo de desarrollo incluso para un primer borrador), Python ocupa un punto intermedio: **es simple de usar pero es un lenguaje de programación real**, con mucha más estructura y soporte para programas grandes que lo que ofrecen los scripts de shell o los archivos batch — y al mismo tiempo, ofrece bastante más verificación de errores que C.

1.4 2. ¿Por qué existe Python? La motivación original

Python nació para resolver una frustración concreta y muy específica del desarrollo de software profesional. Si trabajas con varias bibliotecas en C/C++/Java, el ciclo habitual de escribir/compilar/probar/recompilar puede resultar demasiado lento. Si estás escribiendo un conjunto de pruebas (*test suite*) para una de esas bibliotecas, escribir el código de prueba puede volverse una tarea tediosa. O quizás escribiste un programa que podría beneficiarse de un lenguaje de extensión, pero no quieres diseñar e implementar un lenguaje completamente nuevo solo para esa aplicación.

Python, según la documentación oficial, **es justamente el lenguaje para esos casos**.

1.5 3. Características que distinguen a Python como lenguaje

La documentación oficial atribuye a Python las siguientes características distintivas, descritas con precisión técnica (sin entrar todavía en cómo se usan, eso corresponde a guías futuras):

Lenguaje de muy alto nivel (*very-high-level language*): Python tiene tipos de datos de alto nivel incorporados de fábrica, como arreglos flexibles y diccionarios. Por sus tipos de datos más generales, Python es aplicable a un dominio de problemas mucho más amplio que Awk o incluso Perl, aunque muchas cosas son al menos igual de sencillas en Python que en esos lenguajes.

Modularidad: Python permite dividir un programa en módulos que pueden reutilizarse en otros programas Python. Viene con una amplia colección de módulos estándar que se pueden usar como base para tus propios programas, o como ejemplos para empezar a aprender a programar. Algunos de esos módulos proveen funcionalidades como E/S de archivos, llamadas al sistema, sockets, e incluso interfaces a kits de herramientas de interfaz gráfica de usuario como Tk.

Lenguaje interpretado: Python es un lenguaje interpretado, lo cual puede ahorrar tiempo considerable durante el desarrollo de un programa porque no es necesaria ninguna compilación ni enlazado (*linking*). El intérprete puede usarse de forma interactiva, lo que facilita experimentar con características del lenguaje, escribir programas desechables, o probar funciones durante un desarrollo de programa de abajo hacia arriba (*bottom-up*). También funciona, según la propia documentación oficial, como una práctica calculadora de escritorio.

Cada una de estas características —tipos de datos de alto nivel, módulos, naturaleza interpretada— se desarrollará con mucho más detalle en las guías siguientes (variables y expresiones, objetos, funciones). Aquí solo se presentan como rasgos definitorios del lenguaje, tal como los describe su documentación oficial.

1.6 4. Por qué los programas en Python son compactos y legibles

La documentación oficial explica que Python permite escribir programas de forma compacta y legible, y que los programas escritos en Python son típicamente mucho más cortos que programas equivalentes en C, C++ o Java, por varias razones explícitas:

- Los tipos de datos de alto nivel permiten expresar operaciones complejas en una sola sentencia (*statement*).
- El agrupamiento de sentencias se hace mediante **indentación**, en vez de llaves de apertura y cierre.

- No son necesarias declaraciones de variables ni de argumentos.

Estos tres puntos son, según la propia documentación, la explicación oficial de por qué el código Python suele "leerse" de forma más directa que el de otros lenguajes de propósito similar. El detalle de cómo funciona cada uno de ellos (qué es exactamente una sentencia, cómo opera la indentación en la práctica, cómo se declaran variables sin declaración explícita) corresponde a la guía 02 ("Variables, expresiones y statements"), no a esta introducción.

1.7 5. Extensibilidad: Python como lenguaje de extensión

Otro rasgo que la documentación oficial destaca explícitamente es que Python es **extensible** (*extensible*): si sabes programar en C, es sencillo añadir una nueva función o módulo incorporado (*built-in*) al intérprete, ya sea para realizar operaciones críticas a la máxima velocidad, o para enlazar programas Python con bibliotecas que solo estén disponibles en forma binaria (como una biblioteca gráfica específica de un proveedor). Una vez que te enganchas de verdad con el lenguaje, puedes enlazar el intérprete de Python dentro de una aplicación escrita en C y usarlo como lenguaje de extensión o de comandos para esa aplicación.

Esta característica es la que explica, por ejemplo, por qué bibliotecas científicas de alto rendimiento (NumPy, pandas, scikit-learn, entre muchas otras que aparecerán en guías posteriores de este recorrido) pueden combinar la facilidad de uso de Python con un núcleo de cómputo escrito en C o C++.

1.8 6. Origen del nombre "Python"

Un dato que la propia documentación oficial menciona explícitamente, y anima a repetir: el lenguaje recibe su nombre del programa de la BBC *Monty Python's Flying Circus*, y **no tiene nada que ver con reptiles**. Hacer referencias a sketches de Monty Python en la documentación no solo está permitido, sino que la documentación oficial lo anima activamente.

1.9 7. Estructura oficial del tutorial de Python

Antes de entrar en instalación y manejo del intérprete, vale la pena tener clara la hoja de ruta completa que sigue el tutorial oficial de Python, tal como la describe su propia introducción: el tutorial presenta varias funcionalidades del lenguaje y del sistema a través de ejemplos, comenzando por expresiones simples, sentencias y tipos de datos, pasando luego por funciones y módulos, y finalmente abordando conceptos más avanzados como excepciones y clases definidas por el usuario.

Esta hoja de ruta oficial **coincide, en su orden general, con la secuencia que ya seguías** en tus guías 02-11 (variables/expresiones/statements → objetos → condicionales → iteraciones → funciones → dataframes → predicción/ML). Esta guía de introducción se ubica, por tanto, exactamente donde la documentación oficial la ubicaría: antes de todo eso.

La documentación también es honesta sobre el método de aprendizaje recomendado: dado que la mejor forma de aprender un lenguaje es usándolo, el tutorial invita a jugar con el intérprete de Python mientras se lee. El siguiente capítulo del tutorial oficial explica la mecánica de uso del intérprete — información que, según la propia documentación, "es bastante mundana, pero esencial para poder probar los ejemplos que se muestran más adelante". Las secciones 9 en adelante de esta guía cubren exactamente esa mecánica.

1.10 8. Versiones de Python y ciclo de vida oficial

Python sigue un calendario de mantenimiento formal definido en PEPs (*Python Enhancement Proposals*) específicos para cada serie de versiones (por ejemplo, PEP 619 para la serie 3.10, PEP 664 para la 3.11). Cada serie pasa por etapas: desarrollo activo, mantenimiento con correcciones de errores regulares, y finalmente una etapa de "solo correcciones de seguridad" antes de su fin de soporte (*end of support*).

Al momento de escribir esta guía, según la página oficial de descargas (python.org/downloads), la serie estable más reciente es **Python 3.14**, con la serie 3.13 también vigente y recibiendo actualizaciones regulares; las series 3.9, 3.10 y 3.11 se encuentran ya en la etapa de "solo correcciones de seguridad", lo que significa que ya no reciben correcciones de errores regulares y, en el caso de algunas, ya no se proveen instaladores binarios nuevos para ellas.

> **Recomendación práctica derivada de esto**: para un entorno nuevo de aprendizaje (como el que construirás en las guías siguientes con conda/mamba), conviene partir de una versión dentro del rango de mantenimiento activo (actualmente 3.12, 3.13 o 3.14), evitando series que ya estén solo en modo de parches de seguridad.

1.11 9. Instalación de Python: documentación oficial

Aunque ya configuraste Miniconda/Anaconda en guías previas —que incluyen su propia distribución de Python integrada—, esta sección documenta la instalación de Python **directamente desde la fuente oficial del lenguaje** (python.org), independiente de cualquier distribución de terceros, siguiendo la página oficial de descargas.

1.11.1 9.1. Dónde descargar

La fuente oficial y única recomendada por el propio proyecto Python es **python.org/downloads**. Esa página lista, para cada serie de versión, su estado de mantenimiento, fecha de primer lanzamiento, fecha de fin de soporte, y el PEP que rige su calendario de lanzamientos.

1.11.2 9.2. Windows

python.org/downloads/windows ofrece actualmente el **Python install manager** como mecanismo recomendado de instalación y gestión de versiones de Python en Windows, además de los instaladores tradicionales `.exe` para cada versión específica (por ejemplo, Python 3.14.x, Python 3.13.x). Cada entrada de versión indica explícitamente compatibilidad de sistema operativo — por ejemplo, la documentación oficial advierte que ciertas versiones recientes de Python 3.13 **no pueden usarse en Windows 7 o anterior**.

1.11.3 9.3. Verificación de firma digital en Windows y macOS

La documentación oficial detalla un mecanismo de verificación de autenticidad específico por plataforma:

- **Windows**: los ejecutables de instalación están firmados digitalmente. Esto puede verificarse viendo las propiedades del archivo ejecutable, en la pestaña de Firmas Digitales (*Digital Signatures*), y confirmando el nombre del firmante. El sujeto completo del certificado oficial es `CN = Python Software Foundation, O = Python Software`

`Foundation, L = Beaverton, S = Oregon, C = US`, y desde el 14 de octubre de 2024 la autoridad certificadora es Microsoft Identity Verification Root Certificate Authority (anteriormente, los certificados eran emitidos por DigiCert). La documentación aclara que algunos ejecutables pueden no estar firmados —notablemente, el comando `pip` por defecto—, ya que no se construyen como parte de Python sino que se incluyen desde bibliotecas de terceros.

- **macOS**: los paquetes instaladores descargables desde python.org están firmados con un certificado Apple Developer ID Installer. Desde Python 3.11.4 y 3.12.0b1 (23 de mayo de 2023), los paquetes instaladores de release se firman con certificados emitidos a la Python Software Foundation.

1.11.4 9.4. Linux: nota sobre el alcance de esta guía

En Kubuntu y Arch Linux, una versión de Python suele venir preinstalada como parte del sistema, o se gestiona vía el gestor de paquetes de la distribución (`apt`/`pacman`) o, como ya documentaste en guías previas, vía Miniconda/Anaconda. La instalación de Python "puro" desde fuente en Linux (compilación desde el código fuente publicado en python.org) es un procedimiento avanzado que excede el alcance de esta introducción; la documentación oficial recomienda, para la mayoría de usuarios de Linux, usar el gestor de paquetes de la distribución o una distribución como Anaconda/Miniconda — que es exactamente la vía que ya tienes configurada.

1.12 10. Verificación de la integridad del instalador

Más allá de la firma digital descrita en la sección anterior, la práctica general (consistente con lo ya documentado en tu guía de instalación de Miniconda/Anaconda) es verificar que el archivo descargado no haya sido alterado ni corrompido, comparando su hash con el valor publicado oficialmente junto al instalador en la página de descargas correspondiente, antes de ejecutarlo.

1.13 Tu primer programa

1. El archivo `.py`.. Los archivos de Python usan la extensión `.py`. Cualquier editor de texto sirve para escribirlos (VS Code, PyCharm, Notepad++, incluso el Bloc de notas).

2. `print()` — tu primera función.. `print()` es la función más básica: muestra texto en la consola. Su sintaxis es:

```
print("lo que quieras mostrar")
```

Las comillas pueden ser simples (`'`) o dobles (`"`) — no importa, mientras abras y cierres con el mismo tipo.

3. El intérprete interactivo.. Puedes ejecutar Python de dos maneras:

Modo	Qué hace	Cuándo usarlo
Script (`python archivo.py`)	Ejecuta todo el archivo	Código largo, programas completos
Interactivo (escribir `python` en terminal)	Ejecuta línea por línea	Probar ideas, experimentar rápido

En el modo interactivo ves >>> (el prompt). Escribes código, presionas Enter, y Python ejecuta esa línea inmediatamente.

4. `exit()` — salir del intérprete.. Cuando estés en el modo interactivo, escribe `exit()` para volver a la terminal normal.

1.13.1 Ejemplos

Mínimo — El clásico "Hola mundo", que todo programador escribe primero:

```
print("Hello, World!")
```

Progresivo — Podemos mostrar múltiples cosas:

```
print("Mi primer programa")
print(42)
print("El resultado es", 10 + 5)
# Salida:
# Mi primer programa
# 42
# El resultado es 15
```

Real — Un script que saluda al usuario (lo veremos con variables más adelante, pero es un caso real de uso):

```
# greet.py
print("Bienvenido a Python")
print("Este es tu primer script")
```

Y lo ejecutas: `python greet.py`

1.14 11. Invocar el intérprete: conceptos generales

A partir de aquí, la guía sigue de cerca el capítulo oficial "Using the Python Interpreter" (*Usando el intérprete de Python*) del tutorial.

1.14.1 11.1. Ubicación habitual del ejecutable

Según la documentación oficial, el intérprete de Python suele instalarse como `/usr/local/bin/python3.14` (ajustando el número de versión según corresponda) en las máquinas donde está disponible; agregar `/usr/local/bin` a la ruta de búsqueda de tu shell de Unix permite iniciarlo escribiendo el comando:

```
python3.14
```

La documentación aclara que, dado que la elección del directorio donde vive el intérprete es una opción de instalación, son posibles otras ubicaciones (por ejemplo, `/usr/local/python` es una alternativa popular); conviene verificar con la administración del sistema o la documentación de la propia distribución si hay dudas.

> **Nota oficial relevante para Kubuntu/Arch**: en Unix, el intérprete de Python 3.x **no se instala por defecto** con el ejecutable llamado simplemente `python`, precisamente para no entrar en conflicto con una instalación simultánea de un ejecutable de Python 2.x. Esta es la razón oficial por la que, en muchos sistemas Linux, el comando correcto suele ser `python3` y no `python`.

1.14.2 11.2. En Windows

En máquinas Windows donde instalaste Python desde la Microsoft Store, el comando `python3.14` (o el número de versión correspondiente) estará disponible. Si tienes instalado el lanzador `py.exe`, puedes usar el comando `py`. La documentación oficial remite a la sección sobre el *Python install manager* para conocer otras formas de iniciar Python en Windows.

1.14.3 11.3. Comportamiento general, similar al shell de Unix

El intérprete opera de forma parecida al shell de Unix: cuando se le llama con la entrada estándar conectada a un dispositivo tty, lee y ejecuta comandos de forma interactiva; cuando se le llama con un argumento de nombre de archivo, o con un archivo como entrada estándar, lee y ejecuta un *script* desde ese archivo.

1.15 12. El modo interactivo

Cuando los comandos se leen desde un tty, se dice que el intérprete está en **modo interactivo** (*interactive mode*). En este modo, solicita el siguiente comando con el **prompt primario**, normalmente tres signos de mayor que (`>>>`); para líneas de continuación, solicita con el **prompt secundario**, por defecto tres puntos (`...`).

El intérprete imprime un mensaje de bienvenida indicando su número de versión y un aviso de copyright antes de mostrar el primer prompt:

```
$ python3.14
Python 3.14 (default, April 4 2024, 09:25:04)
[GCC 10.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Las líneas de continuación son necesarias al introducir una construcción de varias líneas. El ejemplo que la propia documentación oficial usa para ilustrarlo (sin que esto sea todavía una explicación de condicionales, que corresponde a la guía 04) es el siguiente:

```
>>> the_world_is_flat = True
>>> if the_world_is_flat:
...     print("Be careful not to fall off!")
...
Be careful not to fall off!
```

Para esta guía, lo relevante de ese ejemplo no es *qué hace* el `if` (eso es tema de una guía futura), sino *cómo se ve* la interacción con el intérprete: el prompt primario `>>>` para la primera línea, el prompt secundario `...` para las líneas de continuación, y una línea en blanco con solo `...` para indicarle al intérprete que el bloque de varias líneas ha terminado.

1.16 13. Edición de línea de comandos en el intérprete

Las funciones de edición de línea del intérprete incluyen edición interactiva, sustitución de historial, y autocompletado de código en la mayoría de los sistemas, según la documentación oficial.

La forma más rápida de comprobar si tu sistema soporta edición de línea de comandos es escribir una palabra en el prompt de Python y luego presionar la flecha izquierda (o `Control-b`). Si el cursor se mueve, tienes edición de línea de comandos disponible. Si no ocurre nada visible, o aparece una secuencia como `^[[D` o `^B`, la edición de línea de comandos no está disponible, y solo podrás usar la tecla de retroceso (*backspace*) para borrar caracteres de la línea actual.

1.17 14. Ejecutar Python como script (modo no interactivo)

Cuando se le da al intérprete un argumento con el nombre de un archivo, lee y ejecuta ese archivo como un *script*, en vez de entrar en modo interactivo. Esta es la forma habitual de ejecutar programas Python guardados en un archivo `.py`:

```
python3 mi_programa.py
```

Esta guía no desarrolla todavía el contenido que iría dentro de `mi_programa.py` —eso corresponde a partir de la guía 02 en adelante—, pero sí establece la mecánica de invocación, que es información de esta etapa "previa al código" según la propia estructura del tutorial oficial.

1.18 15. La opción `-c`: ejecutar un comando directo

Una segunda forma de iniciar el intérprete es `python -c comando [arg] ...`, que ejecuta la(s) sentencia(s) en *comando*, de forma análoga a la opción `-c` del shell. Dado que las sentencias de Python suelen contener espacios u otros caracteres especiales para el shell, la documentación oficial recomienda encerrar *comando* entre comillas en su totalidad.

```
python3 -c "print('hola')"
```

1.19 16. La opción `-m`: ejecutar un módulo como script

Algunos módulos de Python también son útiles como scripts. Pueden invocarse usando `python -m módulo [arg] ...`, lo cual ejecuta el archivo fuente correspondiente a *módulo* como si hubieras escrito su nombre completo en la línea de comandos.

```
python3 -m venv mi_entorno
```

(el ejemplo anterior es solo ilustrativo de la sintaxis `-m`; el manejo de entornos virtuales con `venv` no es tema de esta guía, ya que tu flujo de trabajo usa conda/mamba, documentado en guías previas).

1.20 17. La opción `-i`: modo interactivo después de un script

Cuando se usa un archivo de script, a veces es útil poder ejecutar el script y luego entrar en modo interactivo. Esto puede hacerse pasando `-i` antes del script:

```
python3 -i mi_programa.py
```

Todas las opciones de línea de comandos están descritas, según remite la documentación oficial, en la sección "Command line and environment" de la referencia de Python.

1.21 18. Paso de argumentos: `sys.argv`

Cuando el intérprete los reconoce, el nombre del script y los argumentos adicionales que le siguen se convierten en una lista de cadenas de texto y se asignan a la variable `argv` dentro del módulo `sys`. Se puede acceder a esa lista ejecutando `import sys`.

Algunos detalles precisos de la documentación oficial sobre esta variable:

- La longitud de la lista es de al menos uno; cuando no se da ningún script ni argumentos, `sys.argv[0]` es una cadena vacía.
- Cuando el nombre del script se da como `'-'` (es decir, entrada estándar), `sys.argv[0]` se fija como `'-'`.
- Cuando se usa `-c comando`, `sys.argv[0]` se fija como `'-c'`.
- Cuando se usa `-m módulo`, `sys.argv[0]` se fija con el nombre completo del módulo localizado.
- Las opciones que aparecen después de `-c comando` o `-m módulo` no son consumidas por el procesamiento de opciones del propio intérprete de Python, sino que se dejan en `sys.argv` para que las maneje el comando o módulo correspondiente.

Esta sección se documenta aquí por completitud y rigor (es parte literal del capítulo oficial sobre el intérprete), aunque su uso práctico pleno requiere conceptos de listas y módulos que se desarrollarán en guías posteriores.

1.22 19. El intérprete y su entorno: codificación del código fuente

Por defecto, los archivos fuente de Python se tratan como codificados en **UTF-8**. En esa codificación, pueden usarse simultáneamente caracteres de la mayoría de los idiomas del mundo en literales de cadena, identificadores y comentarios — aunque la biblioteca estándar solo usa caracteres ASCII para identificadores, una convención que cualquier código portable debería seguir. Para mostrar correctamente todos esos caracteres, tu editor debe reconocer que el archivo está en UTF-8 y debe usar una fuente que soporte todos los caracteres del archivo.

> **Nota práctica para tu entorno**: dado que trabajas con Neovim sobre Kubuntu/Arch, UTF-8 es generalmente la codificación por defecto en ambos sistemas, por lo que normalmente no necesitarás declarar nada adicional para escribir comentarios o cadenas en español con tildes y eñes.

1.22.1 19.1. Declarar una codificación distinta a la predeterminada

Para declarar una codificación distinta de la predeterminada, debe agregarse una línea de comentario especial como **primera línea** del archivo, con la siguiente sintaxis:

```
# -*- coding: encoding -*-
```

donde *encoding* es uno de los códecs (`codecs`) válidos soportados por Python. Por ejemplo, para declarar que se use la codificación Windows-1252, la primera línea del archivo fuente debería ser:

```
# -*- coding: cp1252 -*-
```

1.23 20. La línea shebang en Unix/Linux

Una excepción a la regla de "primera línea" es cuando el código fuente comienza con una **línea shebang** de Unix. En ese caso, la declaración de codificación debe agregarse como **segunda línea** del archivo. Ejemplo oficial:

```
#!/usr/bin/env python3
# -*- coding: cp1252 -*-
```

La línea shebang (`#!/usr/bin/env python3`) es lo que permite, en Kubuntu y Arch Linux, hacer un script ejecutable directamente (`./mi_script.py`) sin tener que invocar `python3` explícitamente, siempre que el archivo tenga el permiso de ejecución correspondiente (otorgado, por ejemplo, con `chmod +x`).

1.24 21. Salir del intérprete

Escribir un carácter de fin de archivo (*end-of-file*) — `Control-D` en Unix, `Control-Z` en Windows — en el prompt primario hace que el intérprete salga con un estado de salida cero. Si eso no funciona, puedes salir del intérprete escribiendo el siguiente comando:

```
quit()
```

1.25 22. Comentarios en Python

Los comentarios son una de las pocas piezas de sintaxis que sí corresponde introducir aquí, porque son necesarios para leer y escribir cualquier ejemplo de código desde el primer momento, incluso antes de cubrir variables o expresiones en profundidad.

Según la documentación oficial: **los comentarios en Python comienzan con el carácter numeral (#) y se extienden hasta el final de la línea física**. Un comentario puede aparecer al inicio de una línea, o después de espacio en blanco o código, pero **no dentro de un literal de cadena de texto**. Un carácter # dentro de un literal de cadena es simplemente un carácter numeral, no el inicio de un comentario.

Ejemplo oficial textual:

```
# this is the first comment
spam = 1  # and this is the second comment
          # ... and now a third!
text = "# This is not a comment because it's inside quotes."
```

Dado que los comentarios sirven para clarificar el código y no son interpretados por Python, la documentación oficial señala que pueden omitirse al transcribir los ejemplos del propio tutorial — una observación útil para cuando, en guías futuras, copies y adaptes ejemplos de código.

1.26 23. El intérprete como calculadora: una vista previa controlada

El tutorial oficial, inmediatamente después de explicar cómo invocar el intérprete, presenta a Python "actuando como una simple calculadora": puedes escribir una expresión y el intérprete escribirá su valor. Se incluye aquí únicamente como evidencia de la naturaleza interactiva del intérprete —no como introducción a expresiones o tipos numéricos, que se desarrollarán con rigor en la guía 02—, mostrando el ejemplo textual oficial:

```
>>> 2 + 2
4
>>> 50 - 5*6
20
>>> (50 - 5*6) / 4
5.0
>>> 8 / 5  # la división siempre devuelve un número de punto flotante
1.6
```

Lo único que esta guía retiene de este ejemplo, deliberadamente, es la mecánica de interacción (escribes después de >>>, el intérprete responde en la línea siguiente sin prompt) y el hecho documentado de que la división con / siempre devuelve un valor de tipo flotante. El desarrollo completo de tipos de datos, operadores aritméticos, precedencia, y todo lo demás relacionado a expresiones queda, tal como pediste, para la guía 02 ("Variables, expresiones y statements con Python").

1.27 24. PYTHONSTARTUP: personalizar el inicio del modo interactivo

Cuando usas Python de forma interactiva, suele ser útil tener algunos comandos estándar ejecutándose automáticamente cada vez que se inicia el intérprete. Esto puede lograrse fijando una variable de entorno llamada `PYTHONSTARTUP` con el nombre de un archivo que contenga esos comandos de inicio — de forma similar a la funcionalidad `.profile` de los shells de Unix.

Puntos oficiales relevantes sobre este mecanismo:

- Este archivo **solo se lee en sesiones interactivas**, no cuando Python lee comandos desde un script.
- Se ejecuta en el mismo espacio de nombres donde se ejecutan los comandos interactivos, de modo que los objetos que define o importa pueden usarse sin calificación adicional en la sesión interactiva.
- También es posible cambiar los prompts `sys.ps1` y `sys.ps2` desde ese archivo.

Esta funcionalidad se documenta aquí por rigor y completitud frente a la fuente oficial, aunque su configuración práctica (qué poner dentro de ese archivo) requiere conceptos —importación de módulos, asignación de variables— que se desarrollarán en guías posteriores.

1.28 25. Glosario mínimo para esta etapa

Para cerrar esta introducción sin adelantar contenido de guías futuras, este glosario recoge únicamente los términos que aparecieron en esta guía, con la definición estrictamente ligada al alcance ya cubierto:

Término	Definición en el alcance de esta guía
Intérprete	Programa que lee y ejecuta código Python, ya sea de forma interactiva o desde un archivo de script
Modo interactivo	Modo del intérprete activado cuando lee comandos desde un tty; usa los prompts `>>>` y `...`
Script	Archivo de texto con extensión `.py` que el intérprete lee y ejecuta de forma no interactiva
Prompt primario	El símbolo `>>>` que indica que el intérprete espera una nueva instrucción
Prompt secundario	El símbolo `...` que indica que el intérprete espera la continuación de una instrucción de varias líneas
Comentario	Texto que comienza con `#` y se extiende hasta el final de la línea física; ignorado por el intérprete

Término	Definición en el alcance de esta guía
Shebang	Línea inicial de un script Unix (`#!/usr/bin/env python3`) que indica qué intérprete debe ejecutarlo
`sys.argv`	Lista de cadenas de texto con el nombre del script y los argumentos pasados en la línea de comandos

---

1.29 26. Qué sigue: hoja de ruta de las próximas guías

Siguiendo exactamente la secuencia que ya estableciste, y que coincide con la estructura del propio tutorial oficial de Python (sección 7 de esta guía), las siguientes guías cubrirán, en este orden:

1. **Variables, expresiones y statements con Python** — tipos de datos numéricos y de texto, operadores, precedencia, asignación, convenciones de nombres.
2. **Objetos de Python** — el modelo de objetos del lenguaje.
3. **Ejecución condicional con Python** — `if`, `elif`, `else`, operadores de comparación y lógicos.
4. **Iteraciones con Python** — `while`, `for`, control de bucles.
5. **Funciones con Python** — definición, parámetros, retorno de valores, ámbito.
6. **Dataframes con Python** — introducción a pandas.
7. **Predicción y métricas de performance con Python**.
8. **Métodos de Machine Learning para clasificación con Python**.
9. **Métodos de Machine Learning para regresión con Python**.
10. **Validación cruzada y composición del modelo con Python**.

Esta guía no se adelanta a ninguno de esos contenidos; su función es exclusivamente la de establecer, con la documentación oficial como única fuente, qué es Python y cómo se invoca el intérprete, antes de empezar a escribir código con sentido semántico.

1.30 27. Referencias oficiales

- **Python Tutorial — Índice general**: https://docs.python.org/3/tutorial/index.html
- **Python Tutorial — Cap. 1: Whetting Your Appetite**: https://docs.python.org/3/tutorial/appetite.html
- **Python Tutorial — Cap. 2: Using the Python Interpreter**: https://docs.python.org/3/tutorial/interpreter.html
- **Python Tutorial — Cap. 3: An Informal Introduction to Python** (solo sección de comentarios usada en esta guía): https://docs.python.org/3/tutorial/introduction.html
- **Python — Página oficial de descargas**: https://www.python.org/downloads/
- **Python — Descargas para Windows**: https://www.python.org/downloads/windows/
- **PEP 619 — Python 3.10 Release Schedule**: https://peps.python.org/pep-0619/
- **PEP 664 — Python 3.11 Release Schedule**: https://peps.python.org/pep-0664/
- **Python — Glosario oficial**: https://docs.python.org/3/glossary.html

- **Python — Command line and environment (referencia completa de opciones del intérprete)**: https://docs.python.org/3/using/cmdline.html

---

Edison Achalma Economista — Universidad Nacional de San Cristóbal de Huamanga (UNSCH) Ayacucho, Perú ORCID: 0000-0001-6996-3364

1.31 Para cerrar

1.31.1 Errores comunes

- **Olvidar las comillas:** `print(Hola)` → `NameError` (Python cree que `Hola` es una variable que no existe).
- **Comillas desbalanceadas:** `print("Hola')` → `SyntaxError` (mezclaste comillas).
- **Olvidar paréntesis:** `print "Hola"` → `SyntaxError` (en Python 3, `print` es una función y necesita `()`).
- **Indentación incorrecta:** aunque por ahora solo escribimos una línea, en el futuro, los espacios importan. No uses mezcla de espacios y tabulaciones.

1.31.2 Buenas prácticas

- Usa **f-strings** en lugar de concatenar con + (los veremos pronto) — son más legibles.
- Pon **una línea en blanco** entre bloques de código para que sea más legible.
- Escribe nombres de archivos en **minúsculas y sin espacios**: `mi_script.py`, no `Mi Script.py`.
- Comenta tu código con # para explicar partes complejas.

1.31.3 Ejercicio

Básico — Escritura y ejecución

1. Crea un archivo llamado `mi_primer_programa.py`
2. Escribe código que muestre:
    - Tu nombre
    - Tu edad (un número)
    - La frase: “Estoy aprendiendo Python”
3. Guarda el archivo y ejecútalo desde la terminal con `python mi_primer_programa.py`
4. Luego, abre el intérprete interactivo (`python`) y escribe `print("Python es genial")` directamente para ver cómo se ejecuta en el momento.

1.31.4 Síntesis

- **Python es interpretado**: escribes `.py` y lo ejecutas directamente.
- `print()` muestra texto en la consola; es tu herramienta principal para ver resultados.
- **Dos modos**: script (archivo completo) e interactivo (línea por línea con >>>).
- **Extensión obligatoria**: `.py` en todos tus archivos Python.
- **Ejecución**: `python nombre_del_archivo.py` desde la terminal.

2 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. Instalacion De Anaconda
2. Configurar Entorno Virtual Python Anaconda
3. 01 Introducion A La Programacion Con Python
4. 02 Variables Expresiones Y Statements Con Python
5. 03 Objetos De Python
6. 04 Ejecucion Condicional Con Python
7. 05 Iteraciones Con Python
8. 06 Funciones Con Python
9. 07 Dataframes Con Python
10. 08 Prediccion Y Metrica De Performance Con Python
11. 09 Metodos De Machine Learning Para Clasificacion Con Python
12. 10 Metodos De Machine Learning Para Regresion Con Python
13. Visualizacion De Datos Con Python

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!